PES1UG24AM123

Lab 3: Architecture Justification

Student Club Event Ticketing & Budget Portal (SCETBP)

Architecture Selection: We chose a Layered Architecture (Presentation, Business, Data) for the Student Club Event Ticketing & Budget Portal System.

Two scenario-specific reasons

1. Strict approval sequence in one place. Each proposal must pass Faculty Coordinator, Finance Office and Dean in order (FR-001), and QR tickets are issued only after approval (FR-003). The Business layer holds this workflow logic in one set of components (Proposal Mgr, Approval Flow, Payment Svc), so the rule is enforced once instead of being repeated in screens or scripts.
2. Clear split of work and responsibility. Each layer maps to a group of components that can be built and tested separately. This suits a team of four, where each member owns one epic (Epics 1 to 4) and the layers keep the work from overlapping.

Security advantage: Only the Business layer talks to the Database. The User Interface never queries data directly; it calls the Proposal Mgr through the IProposal interface, and the Business layer checks each request. This is where role checks go, such as a Club Lead seeing only their own club's proposals (US-4.4). QR ticket data is also created and checked inside the Business layer, so attendees cannot change ticket records through the UI.

Performance benefit: Separating the QR Ticket component lets venue check-in (NFR-001, under 100 ms) use a direct, indexed lookup on the ticket table, without going through the approval or reporting code. Heavy work such as dashboard reports (FR-004) runs in its own component, so it does not slow down proposal submission during peak periods (NFR-002).

Why not the other styles

- Client-Server: The approval workflow, ticket generation and reporting would all sit on one central server. That server is a single point of failure during registration and venue check-in, which conflicts with the 99.5% uptime target (NFR-002). It also gives no clear internal boundaries, so the team could not split the work cleanly.
- Microservices: Each component would become a separately deployed service. For a single campus system with a one-week sprint cadence and a team of four, the operational cost is too high. Distributed calls would also add network latency to the 100 ms check-in path (NFR-001), and keeping the approval status and budget consistent across services would need extra coordination.

Layered architecture still has costs: extra calls between layers and the possibility of tight coupling. We reduce these by keeping interfaces narrow (for example IProposal, IApproval, IData) and by letting the components talk only through them.